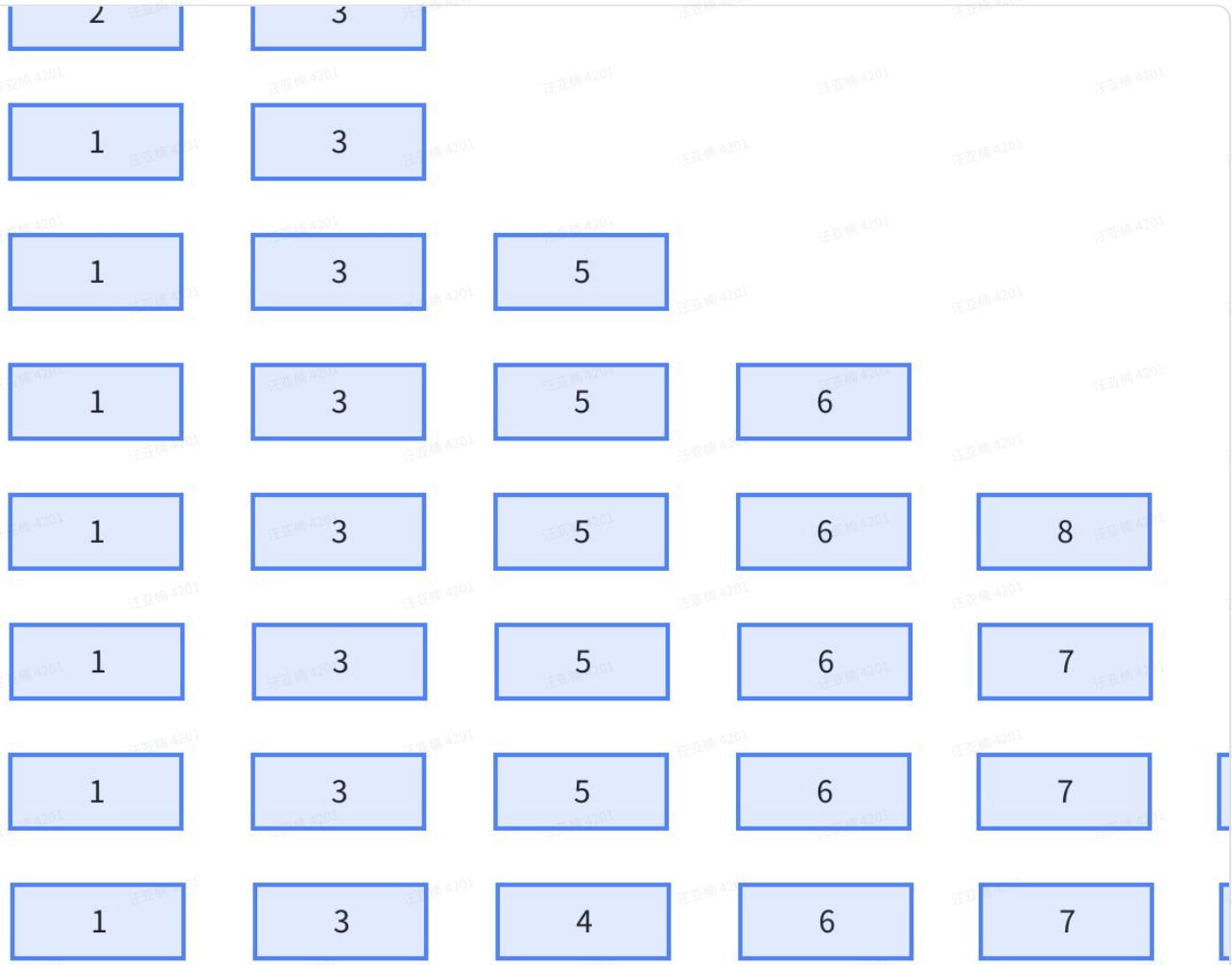
最长递增子序列 + 二分法

```
const arr = [2, 3, 1, 5, 6, 8, 7, 9, 4] // 最长递增子序列: 2 3 5 6 7 9

// 2
// 2 3
// 1 3
// 1 3 5
// 1 3 5 6
// 1 3 5 6 8
// 1 3 5 6 7
// 1 3 5 6 7 9
// 1 3 4 6 7 9 // 到这一步, 值 有可能是错误的, 但个数 一定是正确的

// 默认我们每次放入的时候 我都知道 最小的结尾是谁

// 索引:
// 2 1 8 4 6 7

const getSequence = (arr) => {
  const len = arr.length
  const result = [0] // 索引数组
  const p = arr.slice(0) // 里面内容无所谓和原本的数组相同, 用来存放索引
  let start
  let end
  let middle
  for (let i = 0; i < len; i++) {
    const arrI = arr[i]
    if (arrI !== 0) {
      const resultLastIndex = result[result.length - 1]
      // 取到索引对应的值
      if (arr[resultLastIndex] < arrI) {
        p[i] = resultLastIndex // 标记当前 前一个数对应的索引
        result.push(i) // 当前的值比上一个数大, 直接 push, 并且让这个数记录下它的前一个
        continue
      }

      // 二分查找 找到比当前值大的那一个
      start = 0
      end = result.length - 1
      while (start < end) { // 重合就说明找到了 对应的值
```

```
        middle = ((start + end) / 2) | 0 // 位运算，找到中间位置的前一个（不用位运算，用 Math.floor 也行 Math.floor((start + end) / 2))

        if (arr[result[middle]] < arrI) {
          start = middle + 1
        } else {
          end = middle
        } // 找到 结果集 中，比当前这一项大的数
      }

      // start | end 就是找到的位置
      if (arrI < arr[result[start]]) { // 如果相同或者比当前的还大就不换了
        if (start > 0) {
          p[i] = result[start - 1] // 要将它替换的前一个记住
        }
        result[start] = i
      }
    }
  }

  let resultLen = result.length // 总长度
  let resultLast = result[resultLen - 1] // 取的是最后一个
  while(resultLen-- > 0) {
    result[resultLen] = resultLast
    resultLast = p[resultLast]
  }
  return result
}

// 求当前列表最大递增的个数
// 贪心 + 二分查找

// 在查找中如果当前的比最后一个大，直接插入
// 如果当前这个比最后一个小，采用二分查找的方式，找到已经排好的列表，找到比当前数大的那一项，将其替换

// 1, 8, 5, 3, 4, 9, 7, 6, 2
// 1
// 1 8
// 1 5
// 1 3 4
// 1 3 4 9
// 1 3 4 7
// 1 3 4 6

// 1 2 4 6

```

```
export default getSequence

```